[Snapshot b1ea5fc] Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Introduction

Every year, thousands of students enroll in programming courses, and online judges accumulate millions of submissions to the same set of problems. With the rapid growth of Computer Science enrollment, providing personalized and timely feedback on programming assignments has become a significant challenge, requiring substantial time and effort from faculty (Yi et al. 2017; Hu et al. 2019).

Automated Program Repair (APR) has emerged as a promising solution to this challenge, aiming to deliver automated and personalized feedback to students about their programming errors (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022). Reference-guided APR aligns a buggy program with a correct program for the same problem and transplants the differing region. It uses no model at deployment, so it runs offline and deterministically at no marginal cost per submission. Its applicability depends on the pool of correct programs containing a structurally matching correct program: when the alignment is not possible, the tool reports a structural mismatch and the program is returned unrepaired (Orvalho, Janota, and Manquinho 2025).

More recently, Large Language Models (LLMs) trained on code have shown great potential in repairing programs (Xia and Zhang 2022, 2024; Zhang et al. 2024; Liu et al. 2024; Yang et al. 2024). LLM-based APR does not require a matching reference, so it applies where a structural mismatch would otherwise arise, and it extends beyond introductory exercises to harder problems. What concerns us is how it is deployed. The default is one model query per buggy program, so spending, latency, and dependence on an external service all scale with the number of submissions and recur every term. Queries to the same assignment also share most of their content: hundreds of students attempt one problem and their failures cluster into a few recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019), so the model re-reads one specification and re-derives one mistake many times over. Cheaper tiers and prompt caching lower the price of a call (Anthropic 2025; OpenAI 2024) but leave the number of calls unchanged.

In this paper, we propose CoEP (Cost-Effective Program repair), in which the model writes the repair *tool* rather than the patch generation. The model is invoked once per problem instead of once per submission, and the product of that invocation is not a single repair but an executable program that handles every submission to the problem. An LLM agent receives the problem description, its test cases, and a clustered sample of (buggy, fixed) program pairs drawn from the class, and authors AST rewrite rules named after the misconceptions they repair. These rules run atop a shared symbolic substrate of a generic edit library and reference-guided minimal-hunk transplant from the pool of correct programs. Figure 1 shows one such rule and what it does. The student checks whether the tail of a word is upper case with `a[1:].isupper()`, which is `False` when the tail is empty, so a one-letter word is never converted; the rule the model wrote for this problem tests the slice by equality instead. It repairs 108 of that problem's 560 buggy programs, and the model saw none of them. Every candidate fixed program is validated against the instructor's test cases before it is kept, so the tool can never self-report success, and a defective tool degrades into abstention rather than into a wrong repair. Only the few submissions the tool abstains on escalate to a conventional per-submission call, so coverage is never sacrificed. Model spend therefore moves from a marginal cost that scales with submissions to a fixed cost that scales with problems, and the deployed system retains no model: what a student's submission meets is code that can be read and audited, and that runs offline inside the institution.

Our experiments with 2,970 buggy programs across 11 problems from two corpora, one of course assignments and one of online judge problems, reveal that the synthesized tool outperforms state-of-the-art symbolic repair tools on their own corpus and metric, repairing more programs with smaller patches. Against per-submission LLM repair it matches coverage while modifying less of the student's program, for a fraction of the model spend, and it recovers its one-off build cost within tens of submissions per problem. An ablation separates what each layer contributes: most of the reach comes from the symbolic substrate, and what the

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

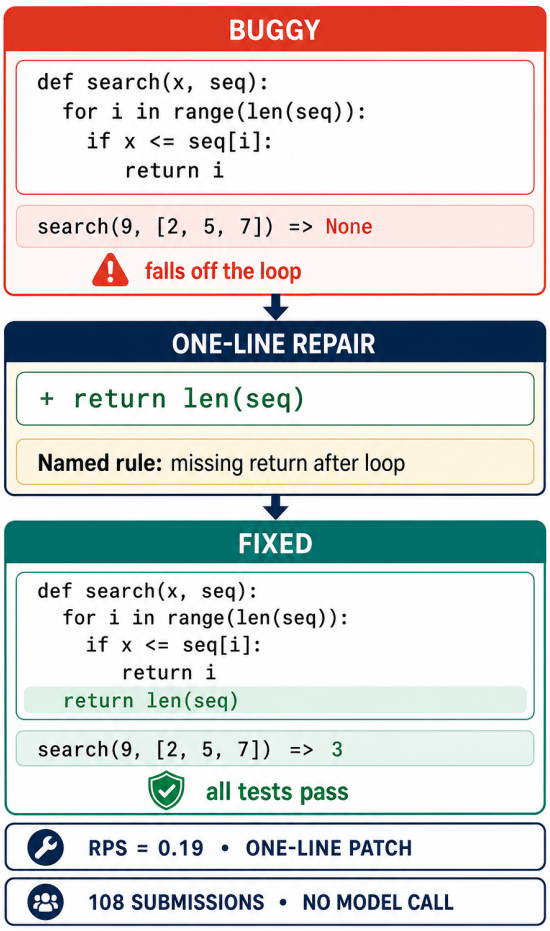


Figure 1: An example of CoEP on the cAPS lOCK problem. The student tests the tail of the word with `isupper()`, which returns `False` for the empty tail of a one-letter word, so `"Z"` is left unchanged. The rule the builder wrote replaces that test with an equality on the slice. It repairs 108 of the problem's 560 buggy programs, none of which the model ever sees.

authored layer adds is smaller patches and a labelled account of which misconception produced each repair. Those two are what a per-submission call cannot amortize, since a call yields a patch but leaves no reusable artifact behind.

In summary, this paper makes the following contributions:

- We reformulate APR so that the model authors an executable repair tool rather than a patch, moving the unit of model invocation from the submission to the problem;
- We present CoEP, which synthesizes misconception-named AST rewrite rules through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023) atop generic edits and reference-guided transplant, validating every patch against the instructor's test cases;
- We evaluate CoEP on real student buggy programs, separate the contribution of each layer with an ablation, and confirm the conclusions agree across two commodity models;

Related Work

APR for programming assignments. Early work reuses the correct programs a class has already submitted. CLARA and Sarfgen align a buggy program with its structurally nearest correct program and transplant the differing blocks (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018), Verifix backs that alignment with formal verification (Ahmed et al. 2022), and Refactory widens the range of alignable programs by refactoring the pool to match the buggy program's structure (Hu et al. 2019). A second line learns the repair function from data, with sequence models for syntactic errors (Gupta et al. 2017), reinforcement learning over edit actions (Gupta, Kanade, and Shevade 2019), and PROSE-based induction of per-assignment transformations from (buggy, fixed) program pairs (Rolim et al. 2017). Most recently, LLMs trained on code address the problem directly: PyDex targets student programs and prepends similar correct programs retrieved by test-case behaviour as few-shot examples (Zhang et al. 2024), ChatRepair adds multi-turn conversation (Xia and Zhang 2024), CREF places a tutor in the loop (Yang et al. 2024), FastFixer fine-tunes for faster per-query inference (Liu et al. 2024), and recent work extends to counterexample-guided minimal repair (Orvalho, Janota, and Manquinho 2025) and learner-tailored retrieval (Dai et al. 2026). Alongside repair itself, OverCode clusters a class's programs to show an instructor which mistakes recurred (Glassman et al. 2015), the same instructor-facing view our rule provenance yields as a by-product of repairing. Existing approaches, symbolic and LLM-based alike, either fix the repair procedure by hand in advance or re-derive it from the model on every submission; neither leaves behind a reusable artifact for the assignment. To remedy this gap, we have the model author that artifact once per assignment, and repair every later submission with it and no model at all.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, P_R)$: a natural-language description d, a set of test cases $T = \{(x_i, y_i)\}$ (function-entry or judge-style stdin/stdout), and the pool P_R of correct programs the class has already submitted, which the setting assumes is available at feedback time, as every per-assignment system from CLARA to Refactory does (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). We use one name per concept throughout. A *submission* is a student program. Submissions divide into *buggy programs* P_B, which fail at least one test case, and *correct programs* P_R, which pass all of them; a correct program used to repair a buggy one is a *reference program*, so P_R is also the pool of reference programs. Repairing the buggy programs yields the *fixed programs* P_F. Individual members are written p_b, p_r and p_f. To *repair* a buggy program $p_b \in P_B$ is to produce a *fixed program* $p_f \in P_F$, and the repair counts as successful only when p_f passes *every* test case in T; passing some of them is not a repair. Patch quality is measured, never gated: RPS, the tree-edit distance between the ASTs of p_b and p_f

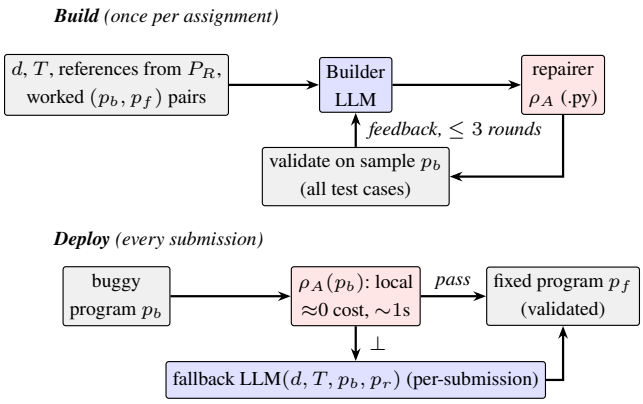


Figure 2: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment most submissions never touch a model.

normalized by the size of p_b, following Refactory's definition (Hu et al. 2019), is reported for every repair, since a small patch is feedback on the *student's* program while a rewrite passes the test cases but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each p_b to a query $\mathrm{LLM}(d, T, p_b)$. CoEP instead synthesizes, once per assignment, a *repairer* ρ_A mapping a buggy program p_b to a fixed program p_f or to $\perp$, an ordinary module with no model access, and deploys the hybrid: $\mathrm{repair}(p_b) = \rho_A(p_b)$ if $\rho_A(p_b) \neq \perp$ (local, $\approx$0 marginal cost), else $\mathrm{LLM}(d, T, p_b, p_r)$: the conventional call, strengthened with one reference program p_r chosen to match the student's strategy by clustering P_R on program structure and drawing p_r from the cluster nearest p_b.

Overview

CoEP runs in two phases (Figure 2). The *build* happens once per assignment: a builder LLM receives d, T, reference programs drawn from P_R, and worked pairs selected from the class's buggy programs, and writes the repairer ρ_A as a Python module. *Deployment* happens on every submission but involves no model: ρ_A takes a buggy program p_b and returns a fixed program p_f or abstains. Only the submissions it abstains on, and those whose local patch is too large, take a conventional per-submission call. That division defines the three configurations we report: $\mathrm{CoEP_L}$ is local repair alone, $\mathrm{CoEP_{LF}}$ adds escalation, and $\mathrm{CoEP_{L+SL}}$ merges the rules self-learning accumulates.

Repairer Synthesis (once per assignment)

Build-set selection. This step is local and costs no model call. The builder does not see all of P_B; it sees a *chosen* sample. We cluster P_B on a structural view, TF-IDF over normalized-AST paths with identifiers folded, and walking clusters largest-first we take from each the member with the smallest tree-edit distance to its nearest reference program, a clean single-misconception exemplar rather than the medoid's typical but tangled program, paired with that reference program. The result is a set of worked (p_b, p_f) pairs, frequent misconceptions first, each shown *with* its minimal repair, so a weak builder needs only to generalize a transformation it is shown rather than infer it.

Rule authoring. The builder LLM receives d, T, reference programs (structural-cluster medoids of P_R), and the worked pairs, and emits the assignment-specific rule module: AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses. The contract is strict. A rule locates its edit on the AST and splices it as a source-span replacement, never re-rendering the module, which would reformat untouched lines. It emits candidates rather than claimed fixes, and it never rewrites a whole function: a rule that cannot express a targeted edit must abstain so the submission escalates.

Generate–validate–refine. A single-shot module can contain its own defects, so the harness closes the loop: it loads the module, runs it on the sample bugs, *independently* validates every candidate patch by executing all test cases, since the repairer's own claims are never trusted, and, while any sample remains unrepaired, returns the module with per-bug outcomes and the still-failing sources for revision, up to $R_{\max}$=3 builder calls. The best-validated round is kept, and build cost is exactly the token count of these exchanges, measured from the raw strings.

Local Repair ($\mathrm{CoEP_L}$)

The frozen tool repairs in two stages, cheapest and smallest-patch first, validating every candidate against all test cases before it is kept.

Named rules and generic search. Rules are *candidate generators* composed by a symptom-ordered search: the failing-test signature ranks rules whose declared symptoms match, and each depth of composition is tested before the next is enumerated, so the shallowest repair is returned by construction. A shared generic library of comparison flips, off-by-one range and index shifts, and Boolean-operator flips is always on, covering common slips regardless of what the builder wrote; it inverts the classical mutation-operator space (Just et al. 2014; Kim et al. 2013; Liu et al. 2019). Every surviving patch carries the ordered rule names behind it, provenance that aggregates into a per-class *misconception profile*.

Reference-guided minimal-hunk transplant. Most student bugs need a structural edit no token-level rule expresses; the pool of correct programs supplies it. We diff p_b against its structurally nearest reference programs, rename the reference program's variables to the student's, matching parameters by position and locals by first-appearance order, and adopt the *smallest* set of differing line-hunks that passes all test cases, candidates ordered smallest-first and batch-validated, returning the minimum-RPS passer. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019), at hunk granularity rather than the block granularity those systems use.

The local path runs on a CPU in $\sim$1–2 s per submission and offline, so no student code leaves the institution. Rule application and transplant are themselves deterministic; the enumeration that feeds them is bounded by a wall clock, so

under heavy load fewer candidates are examined and the tool may abstain where it would otherwise repair. A defective tool degrades to $\perp$, never to a wrong repair, because the harness re-validates every patch before a student sees it.

Escalation ($\mathbf{CoEP_{LF}}$)

A submission both stages abstain on takes the fallback $\mathrm{LLM}(d, T, p_b, p_r)$, its cost counted against CoEP in every comparison. Abstention is not the only trigger. Local repair's acceptance test is the instructor's suite, which says nothing about patch size, so transplant can return a patch larger than the student's whole program while the rule layer must abstain whenever it cannot express a targeted edit. We give transplant the same contract by escalating any passing local patch whose RPS exceeds 1: a patch at least as large as the program it repairs is a rewrite, not a repair, under any assignment's standards. The smaller of the local and model patches is kept, so escalating can cost a call but never coverage or patch size.

The threshold is a constant rather than a quantity calibrated per assignment. We chose it that way because its value is a definition and not a fitted number, and RQ5 reports the sweep over 38 candidate rules that decides whether the choice matters.

Self-Learning ($\mathbf{CoEP_{L+SL}}$)

Each successful fallback repair can be generalized by the LLM into a new named rule, kept only if the augmented tool then repairs that miss on its own. Absorption writes to a staging copy the evaluated pass never consults, so no submission's outcome depends on which preceded it, and the accumulated rules are a separate artifact evaluated by a second frozen pass and priced to themselves (Section). It spends calls only on submissions that already escalated, so it too leaves the build claim untouched.

Experiments

Research Questions.

- **RQ1**: Assignment by assignment, how many submissions does CoEP repair, and how much of each program does it edit?
- **RQ2**: How much does the once-per-assignment structure reduce turnaround and dollars, past what enrollment does the build pay off, and can caching or cheaper tiers close the gap?
- **RQ3**: What produces the repairs, and what does the LLM-authored layer add over the symbolic substrate?
- **RQ4**: Do rules induced from CoEP's own escalations improve the frozen tool on submissions they were not induced from?
- **RQ5**: Does the advantage hold across assignment difficulty, LLM provider, and the escalation threshold?

Baselines. We compare CoEP against the two mainstream lines of program repair for programming assignments: (1) **Refactory** (Hu et al. 2019), which refactors the class's correct programs to match each buggy program's structure and transplants the divergent blocks; on its own corpus we quote its published results, and on Codeforces we run the released tool. (2) **PyDex** (Zhang et al. 2024), which sends each buggy program to the model in its few-shot layout with the three most similar (buggy, correct) exemplars retrieved from the class pool, filters sampled candidates by the test oracle, and returns the survivor closest to the student's program. (3) $\mathbf{PyDex_{PC}}$ re-prices the same queries under provider prompt caching, as an upper bound that assumes a cache hit wherever the provider's rules allow one. CoEP itself is evaluated in three configurations. $\mathbf{CoEP_{L}}$ runs the frozen local tool alone with no deployment LLM use, accepting any patch the test oracle passes, which is the acceptance rule Refactory uses. $\mathbf{CoEP_{LF}}$ adds the per-submission fallback on abstentions and on passing local patches above $\mathrm{RPS} = 1$. $\mathbf{CoEP_{L+SL}}$ re-runs $\mathrm{CoEP_L}$ with the self-learned rules merged and frozen beforehand, so the arm is order-independent; its induction calls are priced to it.

Datasets. We evaluate on 11 problems and 2,970 buggy programs drawn from two real-student corpora: five CS1 assignments contributing 1,783 buggy and 2,442 correct programs at 7.4 test cases and 11.9 lines each, and six Codeforces problems contributing 1,187 buggy and 698 correct at 35.5 test cases and 12.3 lines. Refactory contributes five CS1-level assignments (Hu et al. 2019), the standard corpus of this literature (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025). Codeforces contributes six competitive-programming problems from the open-r1 release (Penedo et al. 2025), spanning difficulty ratings from 800 to 2000, which cover five official rating bands from Newbie to Candidate Master (Codeforces 2024); we keep only problems whose complete official test suites are released, so the oracle is the judge's own. Every submission each corpus labels buggy is a repair target for every arm, so the denominator is the pool as published and is identical across arms. No train/test split is used, as prior per-assignment systems assume, and all results additionally hold with CoEP's build samples excluded from the denominator.

LLM Configuration. Every LLM arm runs on two models, one per provider: **GPT-5.4-mini** ($0.75/$4.50 per Mtok) and **Claude Sonnet 5** ($2/$10 per Mtok). We select each provider's cheapest tier at which prompt caching is functional, in fairness to the caching baseline: $\mathrm{PyDex_{PC}}$ only competes if caching can engage, and we verified that the absolute cheapest tiers rule it out — Haiku 4.5 refuses to cache below its 4,096-token minimum, above these prompts, and GPT-5.4-nano's automatic caching did not engage on identical 1,200-token prompts, whereas both selected models cache at their documented 1,024-token minimums. Cheap tiers also keep every measured dollar gap a conservative floor.

Evaluation Metrics. **Repair rate (RR)** is the percentage of buggy programs a system repairs. A repair counts only when the fixed program passes *all* of the assignment's test cases; a fixed program that passes some of them is not a repair. **Relative patch size (RPS)** is the tree-edit distance between buggy and repaired ASTs normalized by buggy program size, reimplemented from Refactory's definition (Hu et al. 2019); smaller patches preserve the student's approach and

are pedagogically preferable. RPS never *claims* a repair the tests do not support — it enters the pipeline only in CoEP_{LF}, where a value above 1 routes a submission to the fallback and the smaller patch is kept, so the gate can add a call but cannot manufacture a repair or enlarge a patch.

Average turnaround time (ATT) is mean wall-clock seconds per submission, split into ATT_{cpu} (local search) and ATT_{api} (model latency); a system's one-off build wall-clock is an intercept and is reported per assignment. **API cost** is USD of inference over the whole benchmark, including that build, priced at the tiers above from exact token counts. The **escalation rate** f, the fraction of submissions taking a model call, is what the break-even $N^* = C_{\text{build}}/(\Delta(1-f))$ consumes.

Repair rates carry 95% Wilson intervals, and comparisons between arms we ran ourselves are paired by submission and tested with exact McNemar — exact rather than chi-square because the discordant counts are often single-digit. The one exception is Refactory on its own corpus, where we quote published per-assignment results and have no per-submission outcomes to pair; that comparison is marked unpaired wherever it appears.

Setup. All experiments run on one Apple M1 Ultra workstation (64 GB RAM) under Python 3.13, CPU only but for the API calls. No two submissions are evaluated concurrently, so ATT is a measurement rather than a contention artifact; within a submission PyDex issues its $K{=}10$ candidates as one parallel batch, modelling a single $n{=}K$ request, and its ATT is that batch's wall time. Every call is billed and timed. CoEP_{L} and CoEP_{LF} come from a single pass with no duplicated calls. Refactory runs with its original algorithm unchanged, ported to Python 3.

Results

We answer the five research questions stated in the Experimental Setup; together they cover the introduction's claims.

RQ1: Repair Effectiveness The central finding is CoEP's local repair against the strongest non-LLM tool. On the Refactory corpus of 1,783 submissions it repairs **98.48%** at mean **0.289** RPS against Refactory's published **90.80%** at **0.397** — higher coverage and a patch a quarter smaller, at zero marginal LLM cost. That comparison is unpaired, since Refactory's own-corpus results are published aggregates; where we run it ourselves, on Codeforces, the gap widens and the test is paired: 99.92% against 78.60%, exact McNemar $b{=}254$, $c{=}1$, $p{=}8.8{\times}10^{-75}$ over the 1,187 Codeforces submissions. We confine the paired test to that half deliberately: the CS1 rows quote Refactory's published per-assignment figures, so pairing there would silently substitute our own run for theirs. A synthesized tool is not merely cheaper than a symbolic one that consumes every correct program; it repairs more.

Against per-submission LLM repair the separation is not coverage. CoEP_{L} reaches 99.06% and PyDex 98.96, and the paired test finds no difference ($b{=}28$, $c{=}25$, $p{=}0.78$): on CS1-scale programs coverage is near-saturated for any competent repairer (Zhang et al. 2024). The separation is *patch size*, 0.449 against 0.598. Adding the escalation tail takes

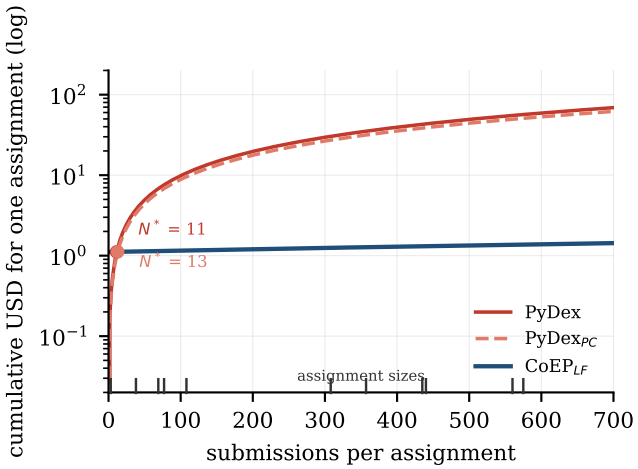


Figure 3: Where amortization overtakes per-submission inference. Ticks mark the eleven assignments' sizes; ten clear N^*.

CoEP_{LF} to **99.66%** at **0.398**, now ahead on coverage too ($b{=}28$, $c{=}7$, $p{=}5.1{\times}10^{-4}$; Wilson $[99.38, 99.82]$ against $[98.52, 99.26]$). Every patch is test-validated before a student sees it, so the only local failure mode is abstention, never a wrong repair, and abstentions take the LLM path.

Table 1 is per-assignment because corpus means hide two things. Refactory's rate swings from 40.3% to 100% while neither LLM arm drops below 93.8%, and the swing does not track difficulty — it repairs all of the rating-2000 problem and four in ten of a rating-1600 one (RQ5). And a low patch size is not always a virtue: Refactory's 0.679 on Codeforces looks competitive against 0.590, but is averaged over the 78.60% it accepts where CoEP_{LF} repairs 100.00%.

RQ1. CoEP's local repair reaches 98.48% RR at 0.289 mean RPS on the Refactory corpus, beating the classical non-LLM tool (90.80%, 0.397) on both coverage and patch size at zero marginal LLM cost. Against per-submission LLM repair, coverage is at parity ($p{=}0.78$) and patch size is a third smaller; escalation adds 99.66% coverage, ahead of PyDex at $p{<}10^{-3}$.

RQ2: Cost Effectiveness and Break-Even The systems differ in *cost structure*, not just constant. A per-submission arm pays an inference on every submission, so its cost grows with enrollment and recurs every term; CoEP pays a one-off C_{build} per assignment, repairs the covered fraction at *zero* marginal LLM cost, and pays per submission only on the escalating tail. Over the benchmark that is \$13.58 against PyDex's \$292.77 at the frontier tier (22×) and \$1.00 against \$28.42 at the cheap one (28×); of our total, \$12.22 is the build and \$1.36 the entire deployment spend across 2,970 submissions.

Fig. 3 turns that into the quantity a department decides on. With Δ the marginal per-submission saving and f the escalation rate, synthesis wins past $N^* = C_{\text{build}}/(\Delta(1-f))$, which our measured build and tail put at **11 submissions per assignment** against PyDex and 13 against PyDex_{PC}. Ten of

	ID	Assignment	Rating	RR (%)				RPS			
				Refactory	PyDex	CoEP_{L}	CoEP_{LF}	Refactory	PyDex	CoEP_{L}	CoEP_{LF}
Refactory	R1	Sequential search	–	99.0	99.5	100.0	100.0	0.390	0.309	0.209	0.206
	R2	Unique dates/months	–	78.2	96.3	93.8	97.7	0.560	0.412	0.396	0.392
	R3	Duplicate elimination	–	97.4	100.0	100.0	100.0	0.340	0.381	0.276	0.276
	R4	Sorting tuples	–	88.2	99.4	100.0	100.0	0.310	0.521	0.337	0.241
	R5	Top-k elements	–	88.0	96.3	100.0	100.0	0.320	0.189	0.190	0.190
Codeforces	C1	Nearly Lucky Number	800	85.2	99.8	100.0	100.0	0.588	0.700	0.595	0.511
	C2	cAPS lOCK	1000	81.4	99.1	100.0	100.0	0.724	1.133	0.732	0.619
	C3	Planning The Expedition	1200	47.8	100.0	98.6	100.0	0.847	0.673	0.637	0.593
	C4	Measuring Lengths in Baden	1400	92.1	100.0	100.0	100.0	0.605	0.623	0.618	0.525
	C5	Jumping Jack	1600	40.3	100.0	100.0	100.0	0.878	0.959	0.932	0.823
	C6	Digital Root	2000	100.0	100.0	100.0	100.0	2.248	1.868	1.527	1.527
	Corpus (2,970 submissions)			**85.9**	**99.0**	**99.1**	**99.7**	**0.500**	**0.598**	**0.449**	**0.398**

Table 1: Per-assignment RR and mean RPS on 2,970 submissions, Sonnet 5 as builder. RPS is averaged over what each arm repairs, so a lower rate beside a lower RPS means an easier subset. All arms see the same inputs, including the 98 that already pass, which CoEP returns unedited. PyDex_{PC} is omitted: caching changes price, not output. Refactory's CS1 rows are published; its Codeforces rows are our run. GPT-5.4-mini is in RQ5.

eleven assignments exceed it, most by an order of magnitude. Below N^* a per-query repairer is the right tool, and we claim nothing else.

Caching and cheaper tiers do not close it. PyDex_{PC} reprices every query under the provider's documented caching rules, as an upper bound assuming a hit wherever one is allowed, and every LLM arm already runs at the cheapest tier at which caching functions. The discount is 10.2% at the frontier tier ($292.77 to $263.05) and 17.3% at the cheap one, and the measurement says why it is small: caching pays for a shared leading *prefix*, and the shared material here is not a prefix. The variable buggy program and retrieved exemplars come first, leaving an identical prefix of about ten tokens; what remains is reuse within one submission across its K candidates. Analytically, N^* is invariant to token price when build and per-query tokens bill on the same schedule, since C_{build} and Δ scale together. The claim is about how many calls per specification are made, not their price — and since every arm runs at the cheapest tier, the measured gaps are a *floor*.

Turnaround is a trade, not a sweep. At the frontier tier CoEP_{LF} answers in 3.06 s against 14.64 s, and the split is the story: we spend 2.69 s on local CPU search and API on a tenth of submissions, PyDex 0.66 s locally and 13.97 s on the network. At the cheap tier the ordering reverses (2.14 s against 3.22 s), because a small model returns one completion faster than our search enumerates and re-executes candidates; we report it rather than quoting only the flattering tier. What the seconds buy is an answer that is rate-limit-free, never leaves the institution, and is unaffected by the deadline-night spikes that make a hosted queue slowest when a class most needs it.

RQ2. CoEP replaces a per-submission inference with a one-off $12.22 synthesis plus a $1.36 tail, 22× cheaper over 2,970 submissions and breaking even at N^*=11 per assignment. Caching recovers only 10.2% and N^* is price-invariant, so the gap is structural.

RQ3: Where the Repairs Come From Two complementary local mechanisms. Named rules make small, labelled edits for common misconceptions — a relaxed comparison, an off-by-one bound — and are surgical, at 25.9% of repaired buggy programs and mean RPS 0.108 (Table 2). The transplant carries the structural repairs instead, and disturbs five times more of the student's code doing it. The transplant lifts a rule-only tool to competitive coverage; the rules keep common cases near-minimal and are the only layer leaving a diagnosis behind. The division is not uniform (Fig. 4): where the pool is deep the transplant dominates, but where one misconception dominates a class the authored layer takes over, as on C4 where a single rule repairs 27 of 38 submissions. The two layers also repair the same misconception differently, which is what keeps the rules' patches small. On C2, 44 of the 140 correct programs test a word's tail with `isupper()` exactly as the buggy programs do, and 42 of those 44 avoid the empty-tail case by adding a separate length guard. A transplant therefore has to import that guard and the structure around it, while the rule Figure 1 shows rewrites the test in place and leaves the student's program otherwise untouched.

What the LLM layer buys. A *framework-only* ablation — the identical tool with the authored rules removed, generic library and transplant left in place — isolates it. Removing them costs **0.37** points of repair rate (99.06 to 98.69) and inflates the mean patch **8%** (0.449 to 0.485). Coverage barely moves because the transplant absorbs what the rules handled, but it absorbs them with a structural edit rather than a targeted one, and the cost lands where the rules were doing real work: C6's patches grow 0.250, C2's 0.110, and R2 is the only assignment losing coverage, by 2.53 points. C4 moves the other way (-0.087), where the transplant finds a smaller edit than the rule that fires there. On a dense correct pool the build therefore buys patch quality and diagnosis rather than reach; the sparse-pool half is RQ5.

The provenance logs say where the effect comes from.

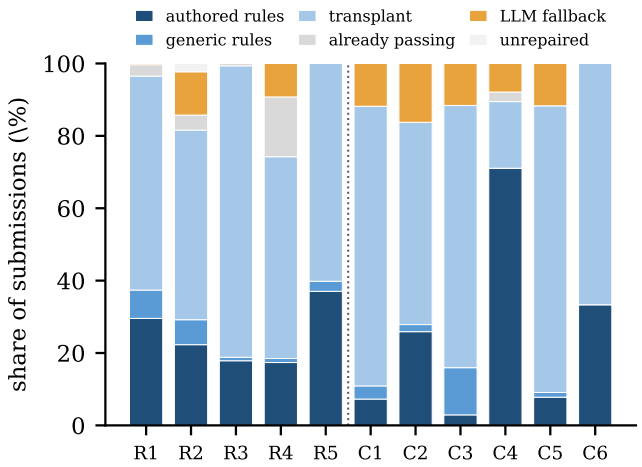


Figure 4: What produced each repair, assignment by assignment. The dotted rule separates the CS1 corpus from Codeforces.

Repair source	share (%)	mean RPS	labelled
LLM-authored rules	21.8	0.124	•
Generic-library rules	4.1	0.025	•
Reference transplant	70.8	0.595	–
Already passing (no edit)	3.3	0.000	–
LLM fallback (CoEP_{LF})	8.5	0.775	–
Framework-only ablation (authored rules removed)			
ablated	98.69	0.485	–
full tool	99.06	0.449	–

Table 2: What produced each repair, its patch size, and whether it carries a misconception label. Shares are over repaired buggy programs; the fallback row is CoEP_{LF}'s and sits outside that denominator. The last rows are the framework-only ablation.

The builder authored 91 rules that fired against the library's 5, firing 789 times against 293 — but only one authored rule fired on more than one assignment, while `generic_flip_compare` alone fired 198 times across 8 of the 11. The authored layer is assignment-specific by construction: it is bought once per assignment and encodes what *this* class got wrong. A fourth producer sits above both — the threshold fires on 317 submissions, 10.8% of those repaired locally, and returns the smaller patch on 232 of them (73.2%); those are 317 of the arm's 345 calls, the other 28 being abstentions.

Diagnosis for free. Every rule-produced patch carries the rule names behind it, so aggregated over a class the tool yields a misconception profile: how often each named mistake occurred, and on which programs. The names read as misconceptions rather than patches — `append_missing_return_len_after_loop`, `count_compare_relax` — and an instructor gets the report at no extra cost from an artifact that can be read and edited. Diagnosis coverage is 25.9% (Table 2); a per-submission API emits none of it at any price.

RQ3. Named rules produce 25.9% of local repairs at mean RPS 0.108, five times smaller than the transplant's 0.595 and each carrying a misconception label; the transplant produces 70.8% and carries coverage. Removing the authored rules costs 0.37 points of coverage and inflates the patch 8%, so on a dense pool the build buys patch quality and diagnosis rather than reach. One of 91 authored rules recurs across assignments while the generic library recurs everywhere — which is why the build is priced per assignment.

RQ4: Learning From Its Own Escalations $\text{CoEP}_{\text{L+SL}}$ is the frozen tool re-run with the rules absorbed from its own fallback repairs. Absorption happens outside the evaluated pass and the augmented tool is frozen before this arm runs, so the comparison against CoEP_{L} is order-independent: the same submissions, the same acceptance rule, two rule sets. Absorption learns only from submissions the local tool fails outright, never from threshold escalations, so its supply of material is exactly the abstention set.

The first result is how little there is to learn from. Across the 22 (assignment, builder) cells the frozen tool abstains on 28 submissions and 19 cells leave nothing to absorb — not a failure of the mechanism but what a 99.06% local repair rate means. Where it does abstain, absorption fires: four rules on R2 with the frontier builder, three on R2 with the cheap one, one on C3.

Where it fires, the rules transfer. R2 moves from 93.79% to **95.40%** on the four, and to **96.55%** on the cheap builder's three; on C3 a single rule, `binarysearch_zero_exit`, closes the last gap to 100%. Paired, both are significant: b=9, c=1, p=0.021 at the frontier tier and b=12, c=0, $p=4.9\times10^{-4}$ at the cheap one. The cheap builder learning *more* is worth stating — the material is the same 27 abstentions, and what differs is how general a rule the model writes from one fix. Eight submissions induced these rules out of 504 evaluated, so the gain is measured almost entirely on programs the rules were never validated against, for $0.76 of induction priced to the artifact. The names again read as misconceptions: `uniqueness_count_gt1_to_neq1`, `counter_var_confusion`.

RQ4. Absorption fires on 3 of 22 cells; on the other 19 the frozen tool leaves no miss to learn from. Where it fires the rules generalize — R2 to 95.40% (frontier) and 96.55% (cheap), C3 to 100%, for $0.76 — significant at p=0.021 and $p<10^{-3}$ on submissions they were not induced from.

RQ5: What the Conclusion Survives **Difficulty.** As the six Codeforces problems climb from rating 800 to 2000 the deep CS1 solution pools give way to sparser ones, and the symbolic baseline tracks that: 90.8% on CS1, 83.1% on the 800–1000 band, **54.3%** on 1200–2000, against CoEP_{LF}'s 99.4/100.0/100.0. But the per-assignment rows say difficulty is not the driver. Refactory repairs all of C6 (rating 2000) and four in ten of C5 (rating 1600); its rate is not monotone in rating. What predicts the collapse is whether the pool holds a structurally matching reference, since block-level

reuse starves when strategies diverge — C5's 77 submissions draw on a pool of two correct programs. Every arm's RPS roughly doubles from CS1 to the hard band, so the ordering survives while the absolute numbers do not.

Provider. CoEP_{LF} reaches 99.7% at 0.398 RPS with the frontier builder and 99.7% at 0.411 with the cheap one — 3% in patch size for an 18-fold difference in build price ($12.22 against $0.68) — with the ordering against both baselines identical either way.

Threshold. We swept 38 candidate rules — median- or mean-anchored with MAD, standard-deviation or interquartile spread, bare quantiles, multiples of the assignment's median, corpus-wide variants, and constants — of which 29 give distinct settings inside the cached escalation prefix and are measured exactly. Three things hold across all. Repair rate is *identical* to four decimals at 99.66%, because the gate re-examines only patches the tests already passed and keeps the smaller. Mean RPS falls with the escalation rate, 0.451 to 0.378, broadly but not monotonically — six adjacent pairs invert, and two settings, `median+3MAD` and `mean*2`, are strictly dominated by the one we ship, buying a larger patch for more calls. No setting on the frontier is an interior optimum, so the choice is a budget rather than a value to discover. Total spend stays within $12.47–$15.61, so every one of the 29 beats all three baselines on repair rate, patch size and dollars at once. We do not claim turnaround across the family: the sweep records calls rather than latency, and RQ2 already reports CoEP_{LF} losing turnaround to PyDex at the cheap tier.

Two findings inside the sweep are negative. Making the threshold relative to the assignment does not pay, since RPS is already normalized by program size and normalizing again directs calls to assignments the tool already handles; and the median-plus-MAD family we began with is Pareto-dominated. We therefore ship the setting whose value is a definition rather than a fitted constant, at the honest cost of being nearly inert where local patches are uniformly small: it fires on 0.0% of R3 against 22.3% of C2.

RQ5. Yes on all three. The symbolic baseline falls 90.8% to 54.3% across the difficulty bands while CoEP_{LF} holds 100%; the two builders agree to 3% in patch size for an 18-fold price difference; and across 29 thresholds the repair rate is invariant while patch size spans 0.378–0.451 and spend $12.47–$15.61.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Codeforces. 2024. Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert; accessed July 2026.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Glassman, E. L.; Scott, J.; Singh, R.; Guo, P. J.; and Miller, R. C. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Transactions on Computer-Human Interaction*, 22(2): 7:1–7:35.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Gupta, R.; Kanade, A.; and Shevade, S. 2019. Deep Reinforcement Learning for Syntactic Error Repair in Student Programs. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence, AAAI 2019*, 930–937. AAAI Press.

Gupta, R.; Pal, S.; Kanade, A.; and Shevade, S. 2017. DeepFix: Fixing Common C Language Errors by Deep Learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence, AAAI 2017*, 1345–1351. AAAI Press.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Rolim, R.; Soares, G.; D'Antoni, L.; Polozov, O.; Gulwani, S.; Gheyi, R.; Suzuki, R.; and Hartmann, B. 2017. Learning Syntactic Program Transformations from Examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, 404–415.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).